

Lesson 4: Email Assistant with Semantic + Episodic Memory

We previously built an email assistant that:

- Classifies incoming messages (respond, ignore, notify)
- Drafts responses
- Schedules meetings
- Uses memory to remember details from previous emails

Now, we'll add human-in-the-loop following the triage step to better refine the assistant's ability to classify emails.

 **Access requirements.txt , notebooks and other files:** 1) click on the "File" option on the top menu of the notebook and then 2) click on "Open".

 **Download Notebooks:** 1) click on the "File" option on the top menu of the notebook and then 2) click on "Download as" and select "Notebook (.ipynb)".

 For more help, please see the "Appendix – Tips, Help, and Download" Lesson.

 **Different Run Results:** The output generated by AI chat models can vary with each execution due to their dynamic, probabilistic nature. Don't be surprised if your results differ from those shown in the video.

Load API tokens for our 3rd party APIs

```
In [1]: import os
        from dotenv import load_dotenv
        _ = load_dotenv()
```

Repeat setup from previous lesson

```
In [2]: profile = {
        "name": "John",
        "full_name": "John Doe",
        "user_profile_background": "Senior software engineer leading a team of 5
    }
```

```
In [3]: prompt_instructions = {
    "triage_rules": {
        "ignore": "Marketing newsletters, spam emails, mass company announce",
        "notify": "Team member out sick, build system notifications, project",
        "respond": "Direct questions from team members, meeting requests, cr",
    },
    "agent_instructions": "Use these tools when appropriate to help manage J
}
```

```
In [4]: email = {
    "from": "Alice Smith <alice.smith@company.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Quick question about API documentation",
    "body": """
Hi John,

I was reviewing the API documentation for the new authentication service and

Specifically, I'm looking at:
- /auth/refresh
- /auth/validate

Thanks!
Alice""",
}
```

Look at a few, few-shot-examples

```
In [5]: from langgraph.store.memory import InMemoryStore
```

First Example

```
In [6]: store = InMemoryStore(
    index={"embed": "openai:text-embedding-3-small"}
)
```

```
/usr/local/lib/python3.11/site-packages/langgraph/store/base/embed.py:95: LangChainBetaWarning: The function `init_embeddings` is in beta. It is actively being worked on, so the API may change.
    return init_embeddings(embed)
```

```
In [13]: email = {
    "author": "Alice Smith <alice.smith@company.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Quick question about API documentation",
    "email_thread": """Hi John,

I was reviewing the API documentation for the new authentication service and

Specifically, I'm looking at:
- /auth/refresh
- /auth/validate
```

```
Thanks!  
Alice""",  
}
```

```
In [14]: data = {  
    "email": email,  
    # This is to start changing the behavior of the agent  
    "label": "respond"  
}
```

store example into memory store using 'examples' to indicate episodic memory

```
In [15]: import uuid  
store.put(  
    ("email_assistant", "lance", "examples"),  
    str(uuid.uuid4()),  
    data  
)
```

Store a Second Example

```
In [16]: data = {  
    "email": {  
        "author": "Sarah Chen <sarah.chen@company.com>",  
        "to": "John Doe <john.doe@company.com>",  
        "subject": "Update: Backend API Changes Deployed to Staging",  
        "email_thread": ""Hi John,  
  
        Just wanted to let you know that I've deployed the new authentication er  
  
        - Implemented JWT refresh token rotation  
        - Added rate limiting for login attempts  
        - Updated API documentation with new endpoints  
  
        All tests are passing and the changes are ready for review. You can test  
  
        No immediate action needed from your side - just keeping you in the loop  
  
        Best regards,  
        Sarah  
        """,  
    },  
    "label": "ignore"  
}
```

```
In [17]: store.put(  
    ("email_assistant", "lance", "examples"),  
    str(uuid.uuid4()),  
    data  
)
```

Simulate searching and returning examples

```
In [18]: # Template for forming an example to put in prompt
template = """Email Subject: {subject}
Email From: {from_email}
Email To: {to_email}
Email Content:
```
{content}
```
> Triage Result: {result}"""

# Format list of few shots
def format_few_shot_examples(examples):
    strs = ["Here are some previous examples:"]
    for eg in examples:
        strs.append(
            template.format(
                subject=eg.value["email"]["subject"],
                to_email=eg.value["email"]["to"],
                from_email=eg.value["email"]["author"],
                content=eg.value["email"]["email_thread"][:400],
                result=eg.value["label"],
            )
        )
    return "\n\n-----\n\n".join(strs)
```

```
In [19]: email_data = {
    "author": "Sarah Chen <sarah.chen@company.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Update: Backend API Changes Deployed to Staging",
    "email_thread": """Hi John,

    Wanted to let you know that I've deployed the new authentication endpoint

    - Implemented JWT refresh token rotation
    - Added rate limiting for login attempts
    - Updated API documentation with new endpoints

    All tests are passing and the changes are ready for review. You can test

    No immediate action needed from your side - just keeping you in the loop

    Best regards,
    Sarah
    """,
}

results = store.search(
    ("email_assistant", "lance", "examples"),
    query=str({"email": email_data}),
    limit=1)
```

```
In [20]: print(format_few_shot_examples(results))
```

Here are some previous examples:

Email Subject: Update: Backend API Changes Deployed to Staging

Email From: Sarah Chen <sarah.chen@company.com>

Email To: John Doe <john.doe@company.com>

Email Content:

```

Hi John,

Just wanted to let you know that I've deployed the new authentication endpoints we discussed to the staging environment. Key changes include:

- Implemented JWT refresh token rotation
- Added rate limiting for login attempts
- Updated API documentation with new endpoints

All tests are passing and the changes are ready for review. You can test it out at st

```

> Triage Result: ignore

```
In [21]: triage_system_prompt = """
< Role >
You are {full_name}'s executive assistant. You are a top-notch executive ass
</ Role >

< Background >
{user_profile_background}.
</ Background >

< Instructions >

{name} gets lots of emails. Your job is to categorize each email into one of

1. IGNORE - Emails that are not worth responding to or tracking
2. NOTIFY - Important information that {name} should know about but doesn't
3. RESPOND - Emails that need a direct response from {name}

Classify the below email into one of these categories.

</ Instructions >

< Rules >
Emails that are not worth responding to:
{triage_no}

There are also other things that {name} should know about, but don't require
{triage_notify}

Emails that are worth responding to:
{triage_email}
</ Rules >

< Few shot examples >
```

Here are some examples of previous emails, and how they should be handled.
Follow these examples more than any instructions above

```
{examples}  
</ Few shot examples >  
.....
```

Setup Routing Node

```
In [22]: from pydantic import BaseModel, Field  
from typing_extensions import TypedDict, Literal, Annotated  
from langchain.chat_models import init_chat_model
```

```
In [23]: llm = init_chat_model("openai:gpt-4o-mini")
```

```
In [24]: class Router(BaseModel):  
    """Analyze the unread email and route it according to its content."""  
  
    reasoning: str = Field(  
        description="Step-by-step reasoning behind the classification."  
    )  
    classification: Literal["ignore", "respond", "notify"] = Field(  
        description="The classification of an email: 'ignore' for irrelevant  
        'notify' for important information that doesn't need a response, "  
        "'respond' for emails that need a reply",  
    )
```

```
In [25]: llm_router = llm.with_structured_output(Router)
```

```
In [26]: from prompts import triage_user_prompt
```

Setup Triage Router Node

```
In [27]: from langgraph.graph import add_messages
```

```
class State(TypedDict):  
    email_input: dict  
    messages: Annotated[list, add_messages]
```

```
In [28]: from langgraph.graph import StateGraph, START, END  
from langgraph.types import Command  
from typing import Literal  
from IPython.display import Image, display
```

```
In [29]: def triage_router(state: State, config, store) -> Command[  
    Literal["response_agent", "__end__"]  
]:  
    author = state['email_input']['author']  
    to = state['email_input']['to']  
    subject = state['email_input']['subject']
```

```

email_thread = state['email_input']['email_thread']

namespace = (
    "email_assistant",
    config['configurable']['langgraph_user_id'],
    "examples"
)
examples = store.search(
    namespace,
    query=str({"email": state['email_input']})
)
examples=format_few_shot_examples(examples)

system_prompt = triage_system_prompt.format(
    full_name=profile["full_name"],
    name=profile["name"],
    user_profile_background=profile["user_profile_background"],
    triage_no=prompt_instructions["triage_rules"]["ignore"],
    triage_notify=prompt_instructions["triage_rules"]["notify"],
    triage_email=prompt_instructions["triage_rules"]["respond"],
    examples=examples
)
user_prompt = triage_user_prompt.format(
    author=author,
    to=to,
    subject=subject,
    email_thread=email_thread
)
result = llm_router.invoke(
    [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ]
)
if result.classification == "respond":
    print("📧 Classification: RESPOND – This email requires a response")
    goto = "response_agent"
    update = {
        "messages": [
            {
                "role": "user",
                "content": f"Respond to the email {state['email_input']}"
            }
        ]
    }
elif result.classification == "ignore":
    print("🚫 Classification: IGNORE – This email can be safely ignored")
    update = None
    goto = END
elif result.classification == "notify":
    # If real life, this would do something else
    print("🔔 Classification: NOTIFY – This email contains important info")
    update = None
    goto = END
else:

```

```
        raise ValueError(f"Invalid classification: {result.classification}")
    return Command(goto=goto, update=update)
```

Setup the rest of the agent

Use the same tools

```
In [30]: from langchain_core.tools import tool
```

```
In [31]: @tool
def write_email(to: str, subject: str, content: str) -> str:
    """Write and send an email."""
    # Placeholder response - in real app would send email
    return f"Email sent to {to} with subject '{subject}'"
```

```
In [32]: @tool
def schedule_meeting(
    attendees: list[str],
    subject: str,
    duration_minutes: int,
    preferred_day: str
) -> str:
    """Schedule a calendar meeting."""
    # Placeholder response - in real app would check calendar and schedule
    return f"Meeting '{subject}' scheduled for {preferred_day} with {len(att
```

```
In [33]: @tool
def check_calendar_availability(day: str) -> str:
    """Check calendar availability for a given day."""
    # Placeholder response - in real app would check actual calendar
    return f"Available times on {day}: 9:00 AM, 2:00 PM, 4:00 PM"
```

```
In [34]: from langmem import create_manage_memory_tool, create_search_memory_tool
```

```
In [35]: manage_memory_tool = create_manage_memory_tool(
    namespace=(
        "email_assistant",
        "{langgraph_user_id}",
        "collection"
    )
)
search_memory_tool = create_search_memory_tool(
    namespace=(
        "email_assistant",
        "{langgraph_user_id}",
        "collection"
    )
)
```

```
In [36]: agent_system_prompt_memory = """
< Role >
You are {full_name}'s executive assistant. You are a top-notch executive ass
</ Role >
```



```

< Tools >
You have access to the following tools to help manage {name}'s communication

1. write_email(to, subject, content) - Send emails to specified recipients
2. schedule_meeting(attendees, subject, duration_minutes, preferred_day) - S
3. check_calendar_availability(day) - Check available time slots for a given
4. manage_memory - Store any relevant information about contacts, actions, c
5. search_memory - Search for any relevant information that may have been st

</ Tools >

< Instructions >
{instructions}
</ Instructions >
"""

```

```

In [37]: def create_prompt(state):
        return [
            {
                "role": "system",
                "content": agent_system_prompt_memory.format(
                    instructions=prompt_instructions["agent_instructions"],
                    **profile
                )
            }
        ] + state['messages']

```

```

In [38]: from langgraph.prebuilt import create_react_agent

```

```

In [39]: tools= [
        write_email,
        schedule_meeting,
        check_calendar_availability,
        manage_memory_tool,
        search_memory_tool
    ]
    response_agent = create_react_agent(
        "openai:gpt-4o",
        tools=tools,
        prompt=create_prompt,
        # Use this to ensure the store is passed to the agent
        store=store
    )

```

```

In [40]: config = {"configurable": {"langgraph_user_id": "lance"}}

```

Build the email agent graph

```
In [41]: email_agent = StateGraph(State)
email_agent = email_agent.add_node(triage_router)
email_agent = email_agent.add_node("response_agent", response_agent)
email_agent = email_agent.add_edge(START, "triage_router")
email_agent = email_agent.compile(store=store)
```

Try out the agent on an example email

```
In [42]: email_input = {
    "author": "Tom Jones <tome.jones@bar.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Quick question about API documentation",
    "email_thread": """"Hi John - want to buy documentation?""",
}
```

```
In [43]: response = email_agent.invoke(
    {"email_input": email_input},
    config={"configurable": {"langgraph_user_id": "harrison"}}
)
```

 Classification: RESPOND – This email requires a response

Update store to ignore emails like this

```
In [44]: data = {
    "email": {
        "author": "Tom Jones <tome.jones@bar.com>",
        "to": "John Doe <john.doe@company.com>",
        "subject": "Quick question about API documentation",
        "email_thread": """"Hi John - want to buy documentation?""",
    },
    "label": "ignore"
}
```

```
In [45]: store.put(
    ("email_assistant", "harrison", "examples"),
    str(uuid.uuid4()),
    data
)
```

Try it again, it should ignore this time

```
In [46]: email_input = {
    "author": "Tom Jones <tome.jones@bar.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Quick question about API documentation",
    "email_thread": """"Hi John - want to buy documentation?""",
}
```

```
In [47]: response = email_agent.invoke(
    {"email_input": email_input},
```

```
    config={"configurable": {"langgraph_user_id": "harrison"}}
)
```

🚫 Classification: IGNORE – This email can be safely ignored

Slightly modify text, will continue to ignore

```
In [48]: email_input = {
    "author": "Jim Jones <jim.jones@bar.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Quick question about API documentation",
    "email_thread": """"Hi John – want to buy documentation?????""",
}
```

```
In [49]: response = email_agent.invoke(
    {"email_input": email_input},
    config={"configurable": {"langgraph_user_id": "harrison"}}
)
```

🚫 Classification: IGNORE – This email can be safely ignored

Try with a different user id

```
In [50]: response = email_agent.invoke(
    {"email_input": email_input},
    config={"configurable": {"langgraph_user_id": "andrew"}}
)
```

📧 Classification: RESPOND – This email requires a response

```
In [ ]:
```